

Jass ML Project

Roadmap to Superhuman Differenzler

April 2026

Current Status

Arena best: 5.8 avg deviation (PIMC 500w + C engine)

Live (fixed): ~10 avg deviation vs Swisslos training bots

Card play: 8/10 - Declaration: 5/10

Target: < 3.5 avg deviation (superhuman)

Architecture

The bot is a Swiss Differenzler Jass AI targeting superhuman play on the Swisslos online platform. Differenzler is a 4-player trick-taking game where each player predicts their score - the goal is to minimize $|\text{predicted} - \text{actual}|$, not maximize points. Total points per round = 157.

Tech Stack

Layer	Technology	Status
Game Engine	C (gcc -O3, bitmask hands, 196K worlds/sec)	Done
Search	PIMC - 500 worlds, C-accelerated	Done
Endgame	Max^n solver, depth 4 (exact last 4 tricks)	Done
Beliefs	Bayesian tracker over opponent declarations	Done
Declaration	Monte Carlo simulation + percentile pick	Needs work
Neural Net	PyTorch 187K params (66% policy accuracy)	POC done
Live Play	Tampermonkey + Flask + WebSocket bridge	Working
Data	Auto-collection pipeline from Swisslos	Working

Live Architecture

```

Swisslos Browser (WebSocket)
  ||
Tampermonkey Extension v7.2.1
  || HTTP (localhost:5000)
Flask Server (server.py v1.1)
  ||
PIMC Engine + C Engine + Belief Tracker + Endgame Solver

```

Performance Evolution

Average deviation measures how far off the bot's prediction is from actual score each round. Lower is better. A perfect round has deviation = 0. Strong humans play around 8-10.

Phase	What Changed	Avg Dev	Speed	Date
1	Clean engine + correct rules + PIMC 25w	9.1	~5s/round	Mar 2026
2	Median declaration + target-aware opponents	7.4	~4s/round	Mar 2026
3	Belief tracking + endgame solver depth 3	6.8	~3.5s/round	Mar 2026
4	C engine 500w + calibration + endgame d4	5.8	36ms/round	Apr 2026
4b	Neural POC (Python rollouts, too slow)	6.0	3s/round	Apr 2026
5	Swisslos deploy (BROKEN seat mapping)	21.3	~12ms/move	Apr 6
5b	Seat fix + 500w + 2000 decl worlds	~10	~20ms/move	Apr 7

The Seat Mapping Bug (Found Apr 7)

The server assumed it was always Swisslos seat 0, but the bot played at seat 2. This caused: wrong point tracking (tracking Computer 1 instead of us), belief updates about ourselves instead of opponents, void detection on wrong players, and wrong opponent points passed to the PIMC search. Fixing this single bug dropped avg deviation from 21.3 to ~10.

Card Play vs Declaration Quality

Post-fix analysis of 8 rounds shows card play is strong (8/10): the bot correctly tracks its target, stops taking tricks when over target, and manages point accumulation well. ALL remaining deviation comes from declaration errors - the simulation opponents don't match how Swisslos bots actually play, causing score predictions to be off.

What Was Tried and Failed

These experiments are documented to avoid repeating them:

Smart C Rollout Policy

5.8 -> 9.1

Added ~150 lines of trick-winning awareness to pick_card. Distorted score distributions used for declarations. Reverted.

ISMCTS (tree search)

6.3-8.0 vs 5.8

5000 iterations spread across 9 moves x 4 players = too thin. PIMC concentrates 500 evaluations per card - more robust.

More PIMC worlds (2000+)

5.9 vs 5.8

Diminishing returns. The heuristic rollout quality is the ceiling, not sample count.

Neural Python Rollouts

6.0 at 3s vs 5.8 at 36ms

66% accuracy but 80x slower than C. The C engine's quantity advantage wins. Fix: embed neural net in C via ONNX.

Percentile Recalibration

9.9 vs 10.2

Sweeping percentile/offset against 8 live rounds. Marginal improvement - the problem is the simulation opponent model, not the percentile.

Roadmap

NEXT

Phase A: Declaration Correction Model

Collect 200+ rounds against Swisslos bots (running overnight). Build a regression model: given hand features (trump count, Nell/Puur, raw points, aces, voids), learn the systematic bias between simulation prediction and actual Swisslos score. Apply as a correction layer on top of the existing declaration model.

Target: 7-8 avg deviation live

1-2 weeks

PLANNED

Phase B: ONNX Neural Rollouts in C

Export the trained PyTorch model to ONNX. Embed ONNX Runtime in jass_engine.c, replacing pick_card with neural inference during rollouts. 500 worlds x neural-quality play instead of heuristic play. This fixes the ROOT CAUSE: simulation opponents play realistically, so declarations become accurate and card play improves further.

Target: 5-6 avg deviation live

2-4 weeks

PLANNED

Phase C: Scaled Training on Real Data

Generate 100K self-play rounds with ONNX-speed neural rollouts. Add 1000+ real Swisslos games as additional training signal. Retrain network with soft labels (per-card PIMC utilities). Real human data teaches patterns that don't exist in self-play.

Target: 4-5 avg deviation live

2-3 weeks after Phase B

FUTURE

Phase D: Opponent Modeling + Meta-game

Cluster opponent play styles from collected data. Neural inference of opponent declarations from play patterns. Meta-game: adjust risk based on cumulative standings (play safe when leading, take risks when behind). Continuous improvement loop: play, collect, retrain, deploy.

Target: < 3.5 avg deviation - SUPERHUMAN

3-4 months total

Timeline to Superhuman

Milestone	Avg Dev	Perfect %	vs Humans	When
Arena baseline (Phase 4)	5.8	20%	(self-play only)	Done
Live deploy + seat fix	~10	~15%	Competitive	Apr 7
Declaration correction (A)	7-8	~20%	Beats most	1-2 weeks
ONNX neural rollouts (B)	5-6	~25%	Beats strong	1-2 months
Scaled training (C)	4-5	~28%	Top tier	2-3 months
Superhuman (D)	< 3.5	~35%	Beats everyone	3-4 months

Key Principles (Validated by Experience)

Speed enables everything.

The C engine's 196K worlds/sec is the foundation. Every improvement multiplies on top.

Consistency > intelligence.

A smarter rollout that distorts distributions is WORSE than a dumb consistent one.

Declaration is half the game.

A perfect card player with a bad declaration still loses.

PIMC beats ISMCTS here.

4-player imperfect info makes tree search iterations too thin.

Rollout quality is the ceiling.

More worlds don't help once the heuristic converges. Neural rollouts are the unlock.

Deploy early, iterate live.

Real-game data is more valuable than theoretical improvements.

What Superhuman Looks Like

The Swisslos Differenzler leaderboard has players with thousands of games. The best humans average around 7-8 deviation over hundreds of rounds. "Superhuman" means consistently beating these players:

Player Type	Avg Deviation	Perfect Rounds	Key Weakness
Casual human	12-15	~5%	Inconsistent declarations
Good human	8-10	~10%	Misreads opponent patterns
Strong human	7-8	~15%	Occasional misjudgments
Our bot (now)	~10	~15%	Declaration model
Our bot (Phase B)	5-6	~25%	Novel situations
Our bot (Phase D)	< 3.5	~35%	None identified

The Endgame

At < 3.5 avg deviation with 35% perfect rounds, the bot would find lines humans miss and exploit tendencies humans can't track. It would adapt to each opponent's style, adjust risk based on match standings, and play optimally from any position. The card play engine is already most of the way there - the remaining journey is about making the prediction model match reality.

The path is clear. Speed got us here. Intelligence gets us the rest of the way.